STM32基础 - RTC实时时钟

课程目标

1. 理解Unix时间戳，掌握时间的变换方法
2. 熟悉并掌握备份寄存器的使用方法
3. 熟悉RTC工作原理，掌握RTC计时方法

课程试验

1. 利用STM32完成读写备份寄存器试验
2. 利用STM32完成实时时钟试验

课堂引入

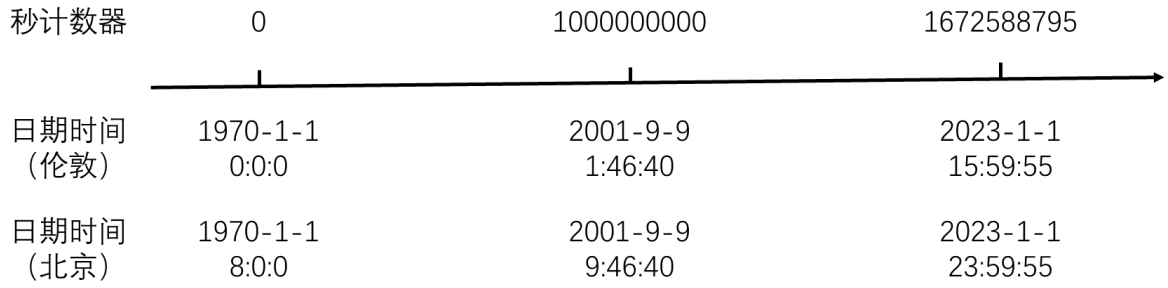
RTC是Real Time Clock的简称，意为实时时钟。RTC模块在电子产品中的应用非常广泛，从大型的工业机器到个人电脑、家用电器甚至是廉价的电子手表，都不乏RTC的身影。BKP备份存储器与RTC配合更是能够让STM32精确管理时间成为可能。

授课进程

一、Unix时间戳

1、基本介绍

Unix时间戳（Unix Timestamp）最早是在Unix系统使用的，所以叫Unix时间戳。之后很多由Unix演变而来的系统，也都继承了Unix时间戳的规定。目前Linux、Windows、MacOS、Android这些系统，它们底层的计时系统都是使用的Unix时间戳，所以在现代计算机世界的底层，Unix时间戳还是在扮演着重要的角色的。Unix 时间戳定义为从UTC/GMT的1970年1月1日0时0分0秒开始所经过的秒数，不考虑闰秒。也就是说时间戳就是一个计数器数值，表示的是一个从1970年1月1日0:0开始到现在总共所经过的秒数，这个值是非常大的。对于人类来说不好记忆和理解，但是它对于计算机处理来说是非常方便的，所以时间戳在计算机程序的底层应用非常广泛。时间戳存储在一个秒计数器中，秒计数器为32位/64位的整型变量。世界上所有时区的秒计数器相同，但是世界上不同地区有不同的时区，全球目前被划分为24个不同的时区，其中穿过英国伦敦的经线被称作本初子午线，前面所说的1970年1月1日0时0分0秒指的就是伦敦时间。对于世界上其他不同时区的地方通过添加偏移来得到当地时间。比如下图就是伦敦时间和北京时间的换算关系。



2、格林尼治和UTC时间

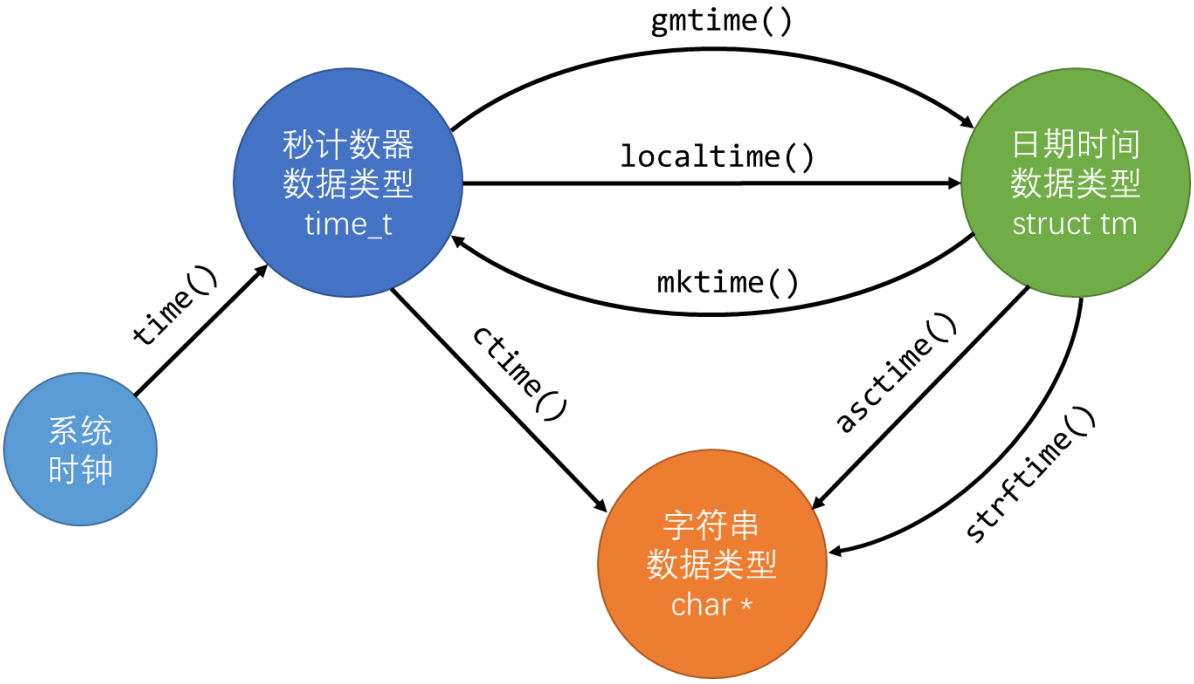
GMT（Greenwich Mean Time）格林尼治标准时间/格林威治标准时间是一种以地球自转为基础的时间计量系统。它将地球自转一周的时间间隔等分为24小时，以此确定计时标准。一天的定义就是地球自转一周，然后一天等分24小时，再等分60分钟，再等分60秒，这样就能确定时间基础了。但是由于地球的自转并不是固定的，现在来说地球自转是越来越慢的状态，按照上面的计时方法来看1秒钟的时间长度其实会越来越长，对于这样一个不固定的时间基准对于现代科学来说是有很大影响的，现代科学需要更加精准的计时，需要一个恒定不变的秒。因此GMT只能算是以前全球计时的时间标准，现在科学家又提出了新的计时系统。

UTC（Universal Time Coordinated）协调世界时。它是一种以原子钟为基础的时间计量系统。它规定铯133原子基态的两个超精细能级间在零磁场下跃迁辐射9,192,631,770周所持续的时间为1秒。原子钟是当前计时最精确的装置，上千万年才误差1秒，所以使用原子钟提供的时间具有定义明确，恒定不变这些好的特性。也就是使用原子钟计时1秒到底是多长，就可以明确的确定了。它定义1秒的时长是和1970年的GMT保持一致的。但是由于地球自转越来越慢，这样长时间计时下去，可能就会发现计时的白天和黑夜颠倒的现象。为了避免这样的现实困扰，所以规定当原子钟计时一天的时间与地球自转一周的时间相差超过0.9秒时，UTC会执行闰秒来保证其计时与地球自转的协调一致。有史以来全球执行闰秒操作已经进行了27次，最近一次闰秒操作的时刻是北京时间2017年1月1日7:59，在下一秒时，时钟会出现7:60，一分钟总共是61秒。通过闰秒操作就能保证UTC既满足科学家研究的需要，又满足人类生活的需要。这是协调世界时的设计思路。UTC是现行的时间标准，它比GMT更加严谨，但是闰秒机制的设计，也给很多程序员带来了无尽的苦恼。因为对照时间戳的定义可以看出时间戳并没有考虑闰秒机制，就是伦敦时间，所以每次产生闰秒时，时间戳的时间就会和国家授时中心的标准时间产生1秒的偏差，这个偏差是需要程序员通过软件来修正的。

3、时间戳转换

C语言的time.h模块提供了时间获取和时间戳转换的相关函数，可以方便地进行秒计数器、日期时间和字符串之间的转换

函数	作用
time_t time(time_t*);	获取系统时钟
struct tm* gmtime(const time_t*);	秒计数器转换为日期时间（格林尼治时间）
struct tm* localtime(const time_t*);	秒计数器转换为日期时间（当地时间）
time_t mktime(struct tm*);	日期时间转换为秒计数器（当地时间）
char* ctime(const time_t*);	秒计数器转换为字符串（默认格式）
char* asctime(const struct tm*);	日期时间转换为字符串（默认格式）
size_t strftime(char, *size_t, const char* , const struct tm*);	日期时间转换为字符串（自定义格式）



4、时间戳转换实例代码

对于上面的方法的作用图表已经描述的很清楚了，具体大家可以通过执行下面的c语言代码再来细细体会。

```c
#include <stdio.h>
#include <time.h>

time_t time_cnt;
struct tm time_date;
char *time_str;

int main() {
    // 获得时间戳方法一
    time_cnt = time(NULL);
    // 获得时间戳方法二
    time(&time_cnt);
    //  time_cnt = 1672688790;
    printf("%d\n", time_cnt);

    // 获得gmt时间
    time_date = *gmtime(&time_cnt);
    printf("%d\n", time_date.tm_year + 1900);
    printf("%d\n", time_date.tm_mon + 1);
    printf("%d\n", time_date.tm_mday);
    printf("%d\n", time_date.tm_hour);
    printf("%d\n", time_date.tm_min);
    printf("%d\n", time_date.tm_sec);
    printf("%d\n", time_date.tm_wday);

    // 获得本地时间
    time_date = *localtime(&time_cnt);
    printf("%d\n", time_date.tm_year + 1900);
    printf("%d\n", time_date.tm_mon + 1);
```

```
    printf("%d\n", time_date.tm_mday);
    printf("%d\n", time_date.tm_hour);
    printf("%d\n", time_date.tm_min);
    printf("%d\n", time_date.tm_sec);
    printf("%d\n", time_date.tm_wday);

    // 依据本地时间计算是正确的，不能依据gmtime计算，结果是错误的。
    // 执行这个函数也会自动计算星期，结构体数据星期结果会回传。
    time_cnt = mktime(&time_date);
    printf("%d\n", time_cnt);

    // 依据时间戳获取时间字符串
    time_str = ctime(&time_cnt);
    printf(time_str);

    // 依据时间结构体获取时间字符串
    time_str = asctime(&time_date);
    printf(time_str);

    char t[50];
    // 自定义时间格式化字符串
    strftime(t, 50, "%Y年%m月%d日 %H:%M:%S", &time_date);
    printf(t);

    return 0;
}
```

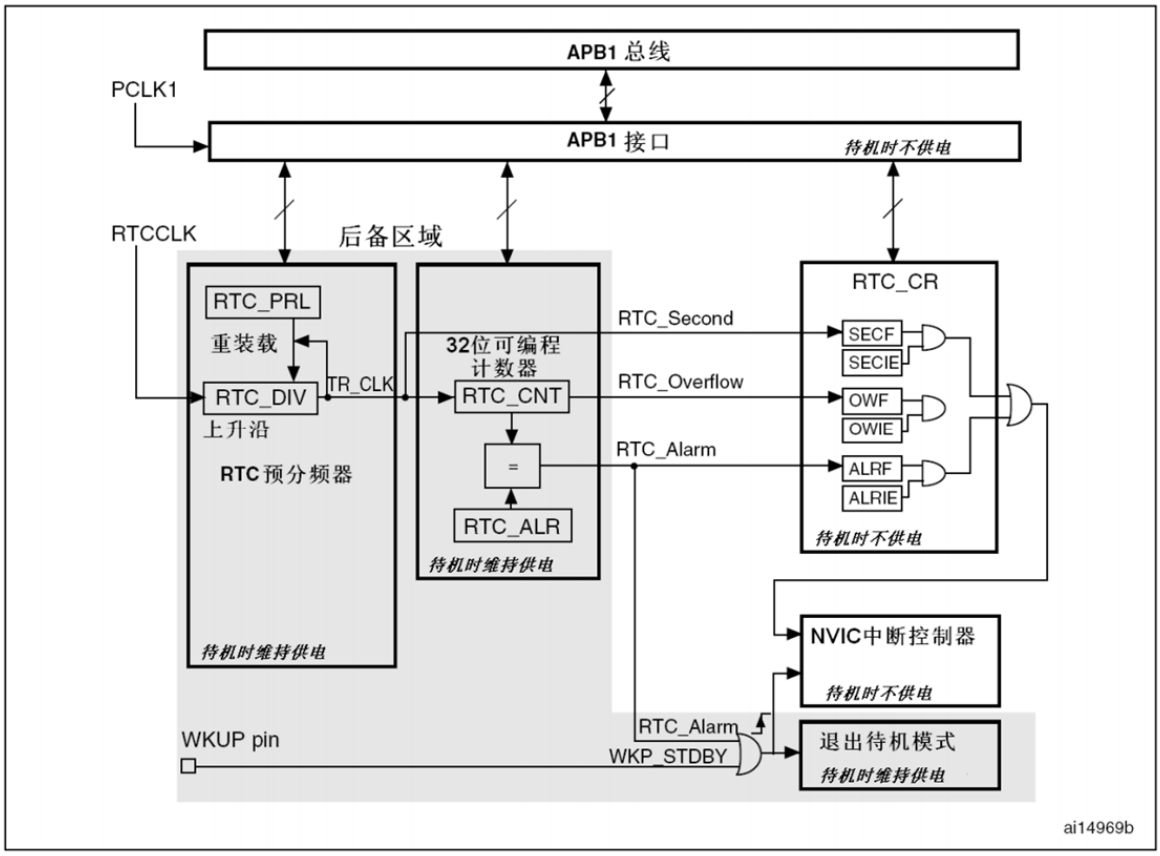
二、RTC

1、基本介绍

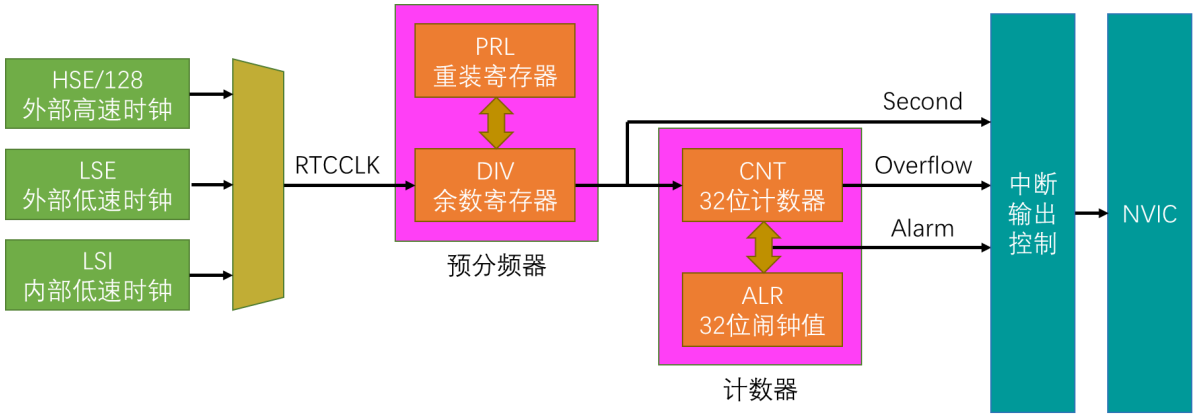
RTC（Real Time Clock）实时时钟。RTC是一个独立的定时器，可为系统提供时钟和日历的功能。RTC和时钟配置系统处于后备区域，系统复位时数据不清零，VDD（2.0-3.6V）断电后可借助VBAT（1.8-3.6V）供电继续走时。这个特性和BKP的是一样的，主电源断电，复位，时间都应该继续，因此就需要备用电池来维持电路的继续运行了。包含一个32位的可编程计数器，可对应Unix时间戳的秒计数器，一个20位的可编程预分频器，可适配不同频率的输入时钟。这里由于一秒计数一次需要一个1Hz的信号驱动计数器，但是实际提供给RTC模块的时钟频率通常都比较高，所以需要分频器来处理转换一下。可选择三种RTC时钟源，分别是HSE时钟除以128（通常为8MHz/128）、LSE振荡器时钟（通常为32.768KHz）、LSI振荡器时钟（40KHz）。之前讲过的RCC时钟树中提到过整个芯片可以有4个时钟源，分别是HSE高速外部时钟信号、HSI高速内部时钟信号、LSI低速内部时钟信号、LSE低速外部时钟信号。其中H表示高速，L表示低速，E表示外部，I表示内部。高速时钟一般供内部程序运行和主要外设使用，低速时钟一般供RTC，看门狗这些东西使用。所以HSE通常是外部主晶振信号，LSE那一路是最为常用的32.768kHz的外部RTC晶振信号，LSI这一路是芯片内部的振荡电路，它的精度不高，只是做备用而已。通常最常用的都是LSE这一路时钟信号。此外还有一点，HSE和LSI两路时钟在主电源断电时，都会断电，只有LSE这一路在主电源断电后，备用电池还会给它供电，所以如果要实现关机状态继续走时的功能就必须选用LSE这一路时钟信号。

2、框图

图154　简化的RTC框图

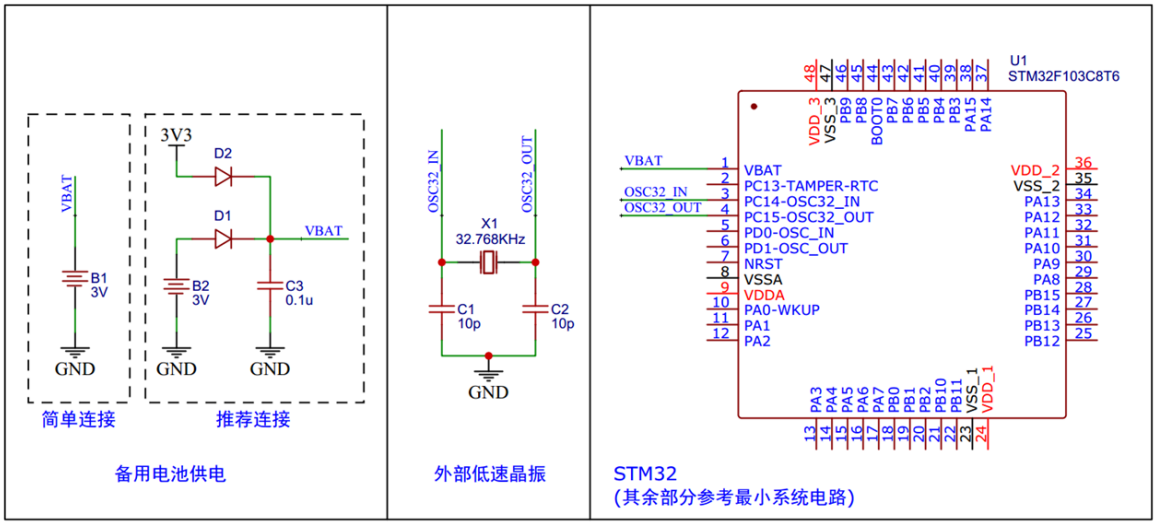


3、基本结构



RTC的基本结构图。它的核心部分如图所示，最左边是RTCCLK时钟来源，需要在RCC里配置。3个时钟选择一个当作RTCCLK之后，RTCCLK先通过预分频器对时钟进行分频，余数寄存器是一个自减计数器，存储当前的计数值。重装寄存器是计数目标，决定分频值，分频之后，得到1Hz的秒计数信号，通向32位计数器，1秒自增一次。下面的32位闹钟值，可以设定闹钟。如果不需要闹钟的话，可以不用管这个。在计数器右边有3个信号可以触发中断，分别是秒信号、计数器溢出信号和闹钟信号。三个信号先通过中断输入控制进中断使能，使能的中断才能通向NVIC，然后向CPU申请中端。在程序中配置最左边的数据选择器，可以选择时钟来源。配置重装寄存器，可以选择分频系数。配置32位计数器，可以进行日期时间的读写。需要闹钟的话，配置30位闹钟值即可。需要中断的话，先允许中断，再配置NVIC，最后写对应的中断函数即可。

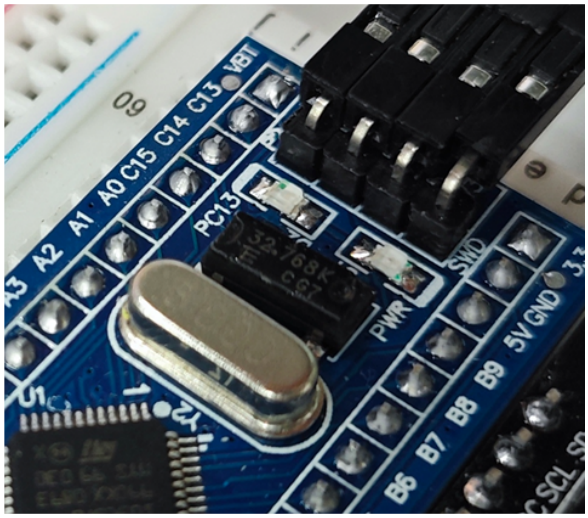
4、硬件电路



为了配合STM32的RTC，还需要一些外部电路的。在最小系统电路上，外部电路还要额外加两部分，第一部分就是备用电池，第二部分就是外部低速晶振。

首先看一下备用电池供电部分。这里给了两个参考电路。第一个是简单连接，就使用一个3伏的电池，负极和系统供地，正极直接接到STM32的VBAT引脚就行了。这个供电方案非常简单，参考来源是STM32的数据手册5.1.6供电方案里就给出来的图。手册中同时还说明了采用VBAT供电的后备电路有32千赫兹振荡器、RTC唤醒电路和后备寄存器。这个设计非常简单，一般来说也没问题。第二种方案是推荐连接。这种连接方法是电池通过二极管D1向VBAT供电，另外主电源的3.3伏，也通过二极管D2向VBAT供电，最后，VBAT再加一个0.1微法的电源滤波电容。这个供电方案的参考来源是STM32的参考设置在4.1.2电池备份区域这一节的描述。其中有几个建议，一个是在某些情况下电流可能通过VDD和VBAT之间的内部二极管注入到VBAT，如果与VBAT连接的电源或者电池，不能承受这样的注入电流，强烈建议在外部VBAT和电源之间连接一个低压降的二极管。另一个是如果在应用中没有外部电池，建议VBT在外部连接到VDD，并连接一个100拉法的陶瓷滤波电容。所以综合这两条建议，设计了推荐连接。电池和主电源都加一个二极管防止电流倒灌，VBAT加一个0.1微法的电源滤波电容。如果没有备用电池，就3.3伏的主电源供电，如果接了备用电池，3.3伏没电时，就是备用电池供电。这两种接线方式的选择，如果只是进行实验，那使用简单连接就行了；如果要画板子设计产品，那还是推荐使用第二种方案的推荐连接，这样更保险。然后再续看一下中间的外部低速晶振部分。这是一个典型的晶振电路。这里X1是一个32.768kHz的RTC晶振，这个晶振不分正负极。两端分别接在OSC32这两个引脚上，然后晶振两端再分别接一个起振电容到GND。这个电路的设计参考来源还是STM32的数据手册，在5.3.6 外部时钟源特性有参考电路。





上面左图中最大的一个是备用电池，备用电池一般可以选择3伏纽扣电池，型号是CR2032。注意这个纽扣电池印制的这一面是正极，上面有个正号标注，另一面比较小的那个电极是负极，这个别搞错了。备用电池旁边的金属壳柱状体是32.768kHz的晶振。这个晶振是比较常见的，过去家里常见的钟表或者电子表拆开经常能见到这种元件，它的全称就是石英晶体振荡器。右图这个是STM32的最小系统板。这个板子自带的有RTC晶振电路，靠近插针黑色的元件上写的有32.768K。这也是一种样式的RTC晶振。然后旁边银白色金属壳柱状体是8mHz的外部高速晶振。不过这个板子没有自带备用电池，VBAT引脚直接通过右上角的这个端口引出来了，如果需要备用电池的话可以接在这个引脚，或者直接把3.3伏的主电源接在这里，也能实现功能。

5、RTC操作注意事项

对于BKP和RTC的开启比较复杂，需要先开启时钟并且设置DBP位才可以开启并允许访问，具体步骤如下：

（1）设置RCC_APB1ENR的PWREN和BKPEN，使能PWR和BKP时钟，

（2）设置PWR_CR的DBP，使能对BKP和RTC的访问。

若在读取RTC寄存器时，RTC的APB1接口曾经处于禁止状态，则软件首先必须等待RTC_CRL寄存器中的RSF位（寄存器同步标志）被硬件置1。这一步对应代码中的一个时钟同步等待库函数，在这里调用一下即可，这一步等待的原因主要是PCLK1和RTCCLK时钟频率不一致，在掉电重新上电后可能会存在时钟不同步，需要等待正确的RTCCLK上升沿到了再读取就没有问题了。

必须设置RTC_CRL寄存器中的CNF位，使RTC进入配置模式后，才能写入RTC_PRL、RTC_CNT、RTC_ALR寄存器。这个步骤由于库函数在写入的时候都已经加了对应操作，所以就不需要管了。

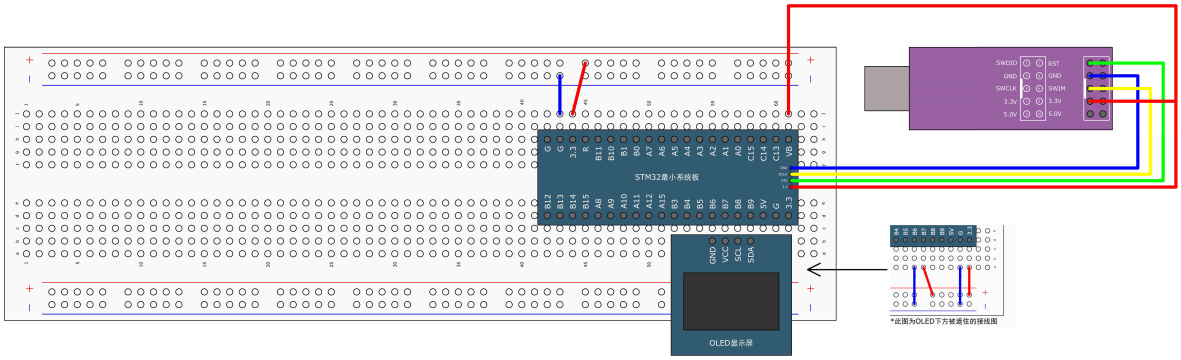
对RTC任何寄存器的写操作，都必须在前一次写操作结束后进行。可以通过查询RTC_CR寄存器中的RTOFF状态位，判断RTC寄存器是否处于更新中。仅当RTOFF状态位是1时，才可以写入RTC寄存器。这一步还是调用一个等待的库函数即可。写入之前需要等待一下，确保数据正确更新。本质原因还是上面说的两个时钟不同步的问题，这些都是细节方面的小问题，了解即可。

三、实时时钟实验

1、实验目的

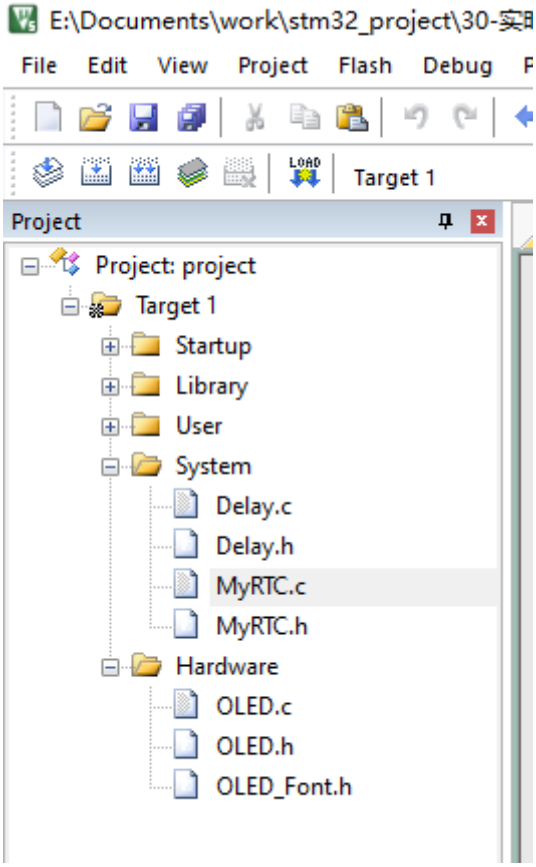
通过控制STM32的RTC外设和备份寄存器，在OLED上显示当前时间数据，在系统复位或者主电源掉电时，都能够正确计时。

2、硬件电路接线图



3、程序设计

工程结构如下图：



其中Hardware组和System组与之前工程不一样，Hardware组只保留了OLED的驱动代码，System组增加了RTC驱动的相关代码。这里注意由于RTC属于STM32的功能，所以没有讲这部分驱动放到外设驱动的Hardware组中，而是放到了System组里。具体代码展示如下：

MyRTC.c

```
#include "stm32f10x.h"                  // Device header
#include <time.h>

uint16_t MyRTC_Time[] = {2023, 1, 1, 23, 59, 55};
void MyRTC_SetTime(void);

void MyRTC_Init(void) {
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_PWR, ENABLE);     // 打开PWR的时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_BKP, ENABLE);     // 打开BKP的时钟

    // 备份寄存器访问使能
    PWR_BackupAccessCmd(ENABLE);

    //  利用备份寄存器中的自定义标志位检查是否首次配置RTC
    if (BKP_ReadBackupRegister(BKP_DR1) != 0xA5A5) {
        // 这里如果外部时钟LSE无法起振，程序会卡死
        // 那么此时下面这句可以改为
        // RCC_LSICmd(ENABLE);
        RCC_LSEConfig(RCC_LSE_ON);  // 打开LSE时钟
        // while (RCC_GetFlagStatus(RCC_FLAG_LSIRDY) != SET);
        while (RCC_GetFlagStatus(RCC_FLAG_LSERDY) != SET);

        // RCC_RTCCLKConfig(RCC_RTCCLKSource_LSI);
        RCC_RTCCLKConfig(RCC_RTCCLKSource_LSE);     // 选择RTCCLK来源为LSE
```

```
        // RTCCLK使能
        RCC_RTCCLKCmd(ENABLE);

        RTC_WaitForSynchro();       // 等待同步
        RTC_WaitForLastTask();      // 等待上一次操作完成

        // LSI时钟频率为40kHz
        // RTC_SetPrescaler(40000 - 1);
        RTC_SetPrescaler(32768 - 1);        // 设置RTC预分频器，预分频后的计数频率为1Hz
        RTC_WaitForLastTask();              // 等待上一次操作完成

        MyRTC_SetTime();        //设置时间，将全局数组里时间值刷新到RTC硬件电路

        BKP_WriteBackupRegister(BKP_DR1, 0xA5A5);   // 在备份寄存器写入自定义标志位，用于判断RTC是否首次执行配置
    } else {
        RTC_WaitForSynchro();       // 等待同步
        RTC_WaitForLastTask();      // 等待上一次操作完成
    }
}

void MyRTC_SetTime(void) {
    time_t time_cnt;        // 定义秒计数器
    struct tm time_date;    // 定义日期时间

    time_date.tm_year = MyRTC_Time[0] - 1900;   // 将数组的时间赋值给日期时间结构体
    time_date.tm_mon = MyRTC_Time[1] - 1;
    time_date.tm_mday = MyRTC_Time[2];
    time_date.tm_hour = MyRTC_Time[3];
    time_date.tm_min = MyRTC_Time[4];
    time_date.tm_sec = MyRTC_Time[5];

    time_cnt = mktime(&time_date) - 8 * 60 * 60;        // 调用mktime函数，将日期时间转换为秒计数器格式
                                                        // 8 * 60 * 60将时间调整为东八区

    RTC_SetCounter(time_cnt);       // 将秒计数器写入到RTC的CNT中
    RTC_WaitForLastTask();          // 等待上一次操作完成
}

void MyRTC_ReadTime(void) {
    time_t time_cnt;        // 定义秒计数器
    struct tm time_date;    // 定义日期时间

    time_cnt = RTC_GetCounter() + 8 * 60 * 60;      // 读取RTC的CNT，获取当前的秒计数器
                                                    // 8 * 60 * 60将时间调整为东八区
    time_date = *localtime(&time_cnt);          // 使用localtime函数，将秒计数器转换为日期时间格式

    MyRTC_Time[0] = time_date.tm_year + 1900;   // 将日期时间结构体赋值给数组的时间
    MyRTC_Time[1] = time_date.tm_mon + 1;
```

```
    MyRTC_Time[2] = time_date.tm_mday;
    MyRTC_Time[3] = time_date.tm_hour;
    MyRTC_Time[4] = time_date.tm_min;
    MyRTC_Time[5] = time_date.tm_sec;
}
```

MyRTC.h

```
#ifndef __MYRTC_H
#define __MYRTC_H

extern uint16_t MyRTC_Time[];

void MyRTC_Init(void);
void MyRTC_SetTime(void);
void MyRTC_ReadTime(void);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "MyRTC.h"

int main(void){

    OLED_Init();
    MyRTC_Init();

    OLED_ShowString(1, 1, "Date:XXXX-XX-XX");
    OLED_ShowString(2, 1, "Time:XX:XX:XX");
    OLED_ShowString(3, 1, "CNT :");
    OLED_ShowString(4, 1, "DIV :");

    while(1){
        MyRTC_ReadTime();       // RTC读取时间，最新的时间存储到MyRTC_Time数组中

        OLED_ShowNum(1, 6, MyRTC_Time[0], 4);
        OLED_ShowNum(1, 11, MyRTC_Time[1], 2);
        OLED_ShowNum(1, 14, MyRTC_Time[2], 2);
        OLED_ShowNum(2, 6, MyRTC_Time[3], 2);
        OLED_ShowNum(2, 9, MyRTC_Time[4], 2);
        OLED_ShowNum(2, 12, MyRTC_Time[5], 2);
        OLED_ShowNum(3, 6, RTC_GetCounter(), 10);   // 显示32位的秒计数器
        OLED_ShowNum(3, 6, RTC_GetDivider(), 10);   //显示余数寄存器
        // 通过这样的变换就可以将余数显示在0-999的毫秒范围内了。
        // OLED_ShowNum(3, 6, (32767 - RTC_GetDivider()) / 32767.0 * 999, 10);
    }
```

```
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是计时时间显示正确，符合预期设计。

课程小结

1. Unix时间戳是从1970年1月1日00:00:00 UTC开始的秒数，它是一种全球统一的时间表示方式。在STM32中，我们可以获取这个时间戳，然后转换为本地时间。
2. 备份寄存器是一个用于存储数据的特殊区域。它可以在系统复位或从待机模式唤醒后，保持数据的完整性和可访问性。这使得它非常适合存储诸如RTC设置、传感器数据等需要长期保存的信息。
3. RTC是一个独立的定时器，拥有一个连续计数的计数器，在相应的软件配置下，可以提供时钟日历的功能。修改计数器的值可以重新设置当前时间和日期。通过和BKP的配合，可以做到在系统复位或从待机模式唤醒后RTC的设置和时间依然维持不变，从而为系统提供精确的时间管理功能。

随堂作业

1. 请阐述RTC基本工作原理